Google Antigravity Architecture

Artifact Lifecycle Management: implementation_plan.md and walkthrough.md

Executive Summary: *In Antigravity,* ***Artifacts*** *serve as the durable, structured state layer connecting autonomous agent reasoning with developer oversight. Rather than flooding chat windows with transient logs, the engineering lifecycle is anchored by two core artifacts: implementation_plan.md (the pre-execution design & alignment contract) and walkthrough.md (the post-execution delivery & verification proof).*

1. The End-to-End Artifact Lifecycle State Machine

Antigravity transitions through distinct operational phases to ensure changes are thoroughly reasoned before modification and verified before completion:

Lifecycle Stage	Active Artifact	Operational Behavior & Transitions
1. Research & Discovery	None / Scratch Notes	Agent performs read-only exploration of repository symbols, patterns, and dependencies. No source code modifications.
2. Plan Formulation & Review Gate	implementation_plan.md	Generates structured plan with RequestFeedback=true. Agent **STOPS** and waits for user approval via the 'Proceed' UI button.
3. Execution & Tool Operation	implementation_plan.md (Living)	Upon user approval, the agent executes file modifications, terminal builds, and subagent delegation against the approved blueprint.
4. Automated Verification	implementation_plan.md	Executes automated unit/integration tests and linters. Self-repairs if failures emerge until all criteria pass.
5. Delivery & Walkthrough	walkthrough.md	Creates/updates the comprehensive walkthrough detailing changes, verification proofs, and visual artifacts for final developer acceptance.

2. Deep Dive: implementation_plan.md (The Pre-Execution Contract)

implementation_plan.md is stored at <appDataDir>/brain/<conversation-id>/implementation_plan.md. It acts as a technical design document aligning the developer and agent on what will be changed.

- **User Review Required:** Uses GitHub alerts ([!IMPORTANT], [!WARNING]) to spotlight breaking changes and key design choices.
- **Proposed Changes Breakdown:** Groups files by subsystem and categorizes each modification explicitly: [NEW], [MODIFY], or [DELETE] with clickable relative links.
- **Verification Plan:** Declares precise automated test commands and manual verification procedures before writing a single line of application code.

```
# [Feature / Goal Description]
## User Review Required
> [!IMPORTANT]
> Database migration drops legacy column `v1_token` in favor of OAuth refresh tokens.

## Proposed Changes
### Backend Authentication Service
#### [MODIFY] [auth_service.py](file:///src/auth/auth_service.py)
#### [NEW] [token_verifier.py](file:///src/auth/token_verifier.py)

## Verification Plan
### Automated Tests: `pytest tests/test_auth.py`
### Manual Verification: Verify OAuth login redirection in local dev server.
```

3. Deep Dive: walkthrough.md (The Delivery Proof)

walkthrough.md is created upon task completion to summarize all accomplishments, test results, and validation proofs.

- **Changes Made:** Concise, high-level summary of modifications linked to source code line ranges.

- **Validation & Test Results:** Command output snippets and test execution summaries proving functionality.
- **Visual Evidence:** Embedded UI screenshots, recordings, or diff carousels illustrating UI state changes.
- **Living Update Rule:** For iterative follow-up tasks, the agent updates the existing walkthrough.md rather than creating disconnected fragments.

```
# Walkthrough - OAuth2 Authentication Migration
## Changes Made
- Updated [auth_service.py](file:///src/auth/auth_service.py#L45-L82) to validate JWT bearer tokens.
- Added [token_verifier.py](file:///src/auth/token_verifier.py) for token rotation.

## Verification Results
- `pytest tests/test_auth.py` -> 14 passed in 1.82s.
- Manual verification on `localhost:3000` confirmed login flow.
```

4. Artifact Metadata & Rich Presentation Engine

Artifacts are governed by explicit metadata attributes that control how the Antigravity UI renders and interacts with them:

Metadata Field	Type	Architectural Role & UI Impact
RequestFeedback	Boolean	When true, blocks execution and renders an interactive 'Proceed' / 'Review' button in the UI for user approval.
UserFacing	Boolean	Distinguishes user-visible documents (plans, reports) from internal scratch scripts stored in brain/<id>/scratch/.
Summary	String	Provides a concise overview of changes for UI preview cards and navigation breadcrumbs in the auxiliary pane.

5. When to Plan vs. When NOT to Plan

- **Always Plan For:** Major architectural changes, ambiguous requirements, multi-file refactoring, database migrations, and non-trivial feature development.
- **Do NOT Plan For:** Investigatory questions ('how does X work?'), simple syntax fixes, minor UI alignments, or direct single-command requests.
- **No Redundant Chat Echoing:** After creating an artifact, the agent does not duplicate the artifact's text in chat; it directs the user to the rendered artifact.